HANDS-ON 4 · DAY 2 / MORNING · GROUP'S SHEET

Lion or tiger: what can 58 photos teach a pretrained network?

The case. A firm wants its photos sorted into classes (products, defects, species) but holds only a few dozen labelled examples of each. Three questions: can it use a network already trained on ImageNet, training only a new head? Can it follow how that training goes? And how far can the test score be trusted?

DATA CARD · LION-TIGER-TRAIN.ZIP, LION-TIGER-TEST.ZIP

What a row is. One photo, a JPEG file in a folder; the network reads its pixels and nothing else. 58 training photos and 57 test photos. The label is the word before the underscore of the file name: `lion_001.jpg`, `tiger_001.jpg`.

The target. `label`, `lion` or `tiger`. Training: 30 lions and 28 tigers. Test: 38 lions and 19 tigers, so always saying `lion` is right 67% of the time, and one test photo is worth 1.75 points.

How the photos were chosen. Taken from the web for a course exercise. No source, photographer or licence is recorded with them. They are not a sample of any population of photos.

Its history. The inherited test set held 65 photos, 8 of them byte-identical copies of training photos: they were removed. Two training photos carried the other animal's label: they were corrected. Every photo was renamed `<label>_<nnn>.jpg`, so that one rule reads all the labels.

Source and licence. Lions and tigers: web photos, licences unknown, used in class and never republished. Cats, after the checkpoint: the cat class of Natural Images, P. Roy, S. Ghosh, S. Bhattacharya and U. Pal (2018), Effects of degradations on deep neural network architectures, arXiv:1807.10108; 120 training and 80 test photos of its 885. The licence of its public copy is not verified.

archive	photos	lions	tigers	cats	used in
`lion-tiger-train.zip`	58	30	28		the folder `train`, steps 1 and 2
`lion-tiger-test.zip`	57	38	19		the folder `test`, step 3
`cats-train.zip`	120			120	`train`, panel 5
`cats-test.zip`	80			80	`test`, panel 5

STEP 1 · LOAD THE PHOTOS AND THEIR LABELS

1 ☐ **Upload the training photos**
In a new project, on the Flow: **+ Dataset** → **Folder**, named `train`; upload the 58 photos unzipped from `lion-tiger-train.zip`.

2 ☐ **List the files**
On the folder `train`: **+ Recipe** → **List Contents**, **Run**. One row per photo, with its `path`.

Rows: _____

3 ☐ **Read the label from the name**
On the listing: **+ Recipe** → **Prepare**, a step **Extract with regular expression** on `path`, pattern `([a-z]+)_[0-9]+`, output column `label`; **Run**.

Lions: _____ tigers: _____

STEP 2 · TRAIN A NEW HEAD ON A PRETRAINED NETWORK

1 ☐ **Create the analysis**
Select the labelled dataset, then **Lab** → **Image classification**: target `label`, the images in the folder `train`.

2 ☐ **Reuse it, do not fine-tune**
Design → **Train/Test set**: **Split the dataset**, train ratio 0.8. **Design** → **Training**: pretrained model **EfficientNet B0**, **Number of fine-tuned layers** 0; leave the epochs at their default. **Train**.

Training time: _____ epochs run: _____

3 ☐ **Follow the training, then read its own test**
Results: the training curve, if shown, then the confusion matrix; click a cell to see its photos in the image feed.

Accuracy: _____ on how many photos? _____

STEP 3 · SCORE PHOTOS IT HAS NEVER SEEN

1 ☐ **Load the test photos**
A second folder `test` with the 57 photos of `lion-tiger-test.zip`; list and label them as in steps 1.2 and 1.3.

2 ☐ **Score them**
In the analysis: **Deploy**, model name `model_lions`. Then on the labelled test listing: **+ Recipe** → **Score**, with that model and the managed folder `test`; **Run**.

3 ☐ **Count the errors**
On the scored dataset, a Prepare step **Formula** into `check`: `if(label == prediction, "right", "wrong")`.

Wrong: _____ of 57 accuracy: _____ lion-always baseline: 67%

What do the wrong photos have in common? Is any of them wrongly labelled?

☐ **Reached this line? Stop here: the debrief reads every group's test score, with its 1.75 points a photo.**

Panels 4 to 6 follow slide 56 of **Deep learning for decisions**.

4 · FINE-TUNE TWO LAYERS, AND COMPARE

1 ☐ **Unlock two layers**
In the same analysis: **Design** → **Training**, **Number of fine-tuned layers** 2, the rest unchanged. **Train**.

fine-tuned layers	training time	accuracy in Results
0 (step 2)		
2		

Both accuracies come from Dataiku's own test split: how many photos is that, and what is one photo worth?

_____

5 · ADD A THIRD, LARGER CLASS: CATS

1 ☐ **Upload the cats**
The 120 photos of `cats-train.zip` into `train`, the 80 of `cats-test.zip` into `test`; rebuild the listings and their Prepare recipes: the pattern reads `cat` too.

2 ☐ **Retrain**
In the same analysis, 0 fine-tuned layers: **Train**. Three classes now.

class	training photos	test photos	recall
lion	30	38	
tiger	28	19	
cat	120	80	

3 ☐ **Read the averages**
Results → **Metrics**: the averaged scores.

Macro average: _____ weighted average: _____

6 · DECIDE

Which model would the firm deploy, with 0 or 2 fine-tuned layers, and why? For the three classes, which score would you report, the macro or the weighted average?
